

Tech Challenge IADT – Fase 5

Instituição: FIAP – Tech Challenge (IADT)

Fase: 5

Projeto: Detecção de Componentes de Arquitetura e Modelagem de Ameaças STRIDE por Visão Computacional

Integrantes do grupo

- **Nome:** Bruno Quadrotti de Freitas
- **RM:** 368164
- **Grupo:** 61

Entregáveis

- **Link do projeto no GitHub:** https://github.com/brunoquadrotti/tech_challenge_fiap_fase5
- **Link do vídeo no YouTube (demonstração):** <https://youtu.be/y3xFsJTmm8k>

1. Introdução

A análise de segurança em arquiteturas de software é uma atividade essencial no ciclo de desenvolvimento, sobretudo quando conduzida ainda na fase de projeto — abordagem conhecida como *security by design*. Nesse contexto, a modelagem de ameaças permite antecipar riscos antes que o software seja implementado, reduzindo o custo de correção e ampliando a robustez das soluções. Entre as metodologias consolidadas para essa finalidade, destaca-se o **STRIDE**, proposto pela Microsoft, que organiza as ameaças em seis categorias: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service* e *Elevation of Privilege*.

Apesar de sua utilidade, a modelagem de ameaças é tradicionalmente um processo manual, dependente da leitura visual de diagramas de arquitetura e da experiência do analista. Diagramas de arquitetura são artefatos predominantemente visuais, o que abre espaço para a aplicação de técnicas de **Visão Computacional** capazes de reconhecer automaticamente os componentes representados.

O presente trabalho propõe o desenvolvimento de uma aplicação capaz de receber a imagem de um diagrama de arquitetura de software, **detectar automaticamente seus componentes** por meio de um modelo supervisionado de detecção de objetos (YOLO) previamente treinado e, a partir dessas detecções, **executar uma análise baseada na metodologia STRIDE**, gerando ao final um relatório em PDF contendo os componentes identificados, os relacionamentos inferidos, as ameaças encontradas e suas respectivas recomendações.

A proposta une **Visão Computacional supervisionada** — responsável exclusivamente pela detecção dos componentes — a uma **engine de regras determinística**, responsável

pela associação das ameaças STRIDE. Essa separação é uma decisão arquitetural central do projeto: a Inteligência Artificial atua unicamente na etapa de detecção, enquanto a modelagem de ameaças permanece transparente, previsível e auditável, baseada integralmente em regras.

Nota: este é um protótipo acadêmico (MVP) desenvolvido para fins de pesquisa e aprendizado, no âmbito do Tech Challenge da Pós-Graduação em Inteligência Artificial para Desenvolvedores (FIAP). Seu objetivo é demonstrar a integração entre Visão Computacional e análise de segurança arquitetural, não substituindo a avaliação de um especialista em segurança.

2. Objetivos do projeto

2.1. Objetivo geral

Projetar e implementar uma aplicação capaz de analisar imagens de diagramas de arquitetura de software, detectar automaticamente seus componentes por meio de um modelo supervisionado de detecção de objetos e, a partir dessas detecções, produzir uma análise de ameaças baseada na metodologia STRIDE, consolidada em um relatório em PDF para apoio à análise de segurança em fase de projeto.

2.2. Objetivos específicos

1. Construir e anotar um dataset de diagramas de arquitetura de software (nuvens AWS e Azure) para detecção supervisionada de componentes arquiteturais.
2. Treinar um modelo de detecção de objetos (YOLO) capaz de reconhecer componentes e limites de confiança (*trust boundaries*) em diagramas.
3. Encapsular a inferência do modelo em um módulo desacoplado, reutilizável pelo restante da aplicação.
4. Implementar um módulo de pré-processamento mínimo, responsável por preparar a imagem para a inferência.
5. Desenvolver uma estratégia de extração de relacionamentos entre os componentes detectados, baseada exclusivamente em informação geométrica.
6. Construir uma base de conhecimento STRIDE em formato JSON, desacoplada do código e facilmente expansível.
7. Implementar uma engine de análise STRIDE 100% baseada em regras, que associa componentes e relacionamentos a ameaças e contramedidas.
8. Gerar um relatório em PDF contendo os componentes detectados, os relacionamentos, as ameaças identificadas e as recomendações.
9. Integrar todo o pipeline em uma interface web construída com Streamlit, com visualização das detecções e download do relatório.

3. Arquitetura da solução

A arquitetura do sistema segue o padrão de **pipeline linear e modular**, no qual cada etapa recebe a saída da etapa anterior e produz uma estrutura de dados bem definida para a etapa seguinte. Esse desenho mantém o acoplamento baixo: cada módulo depende apenas de estruturas de dados simples, e não das implementações concretas dos demais.

3.1. Princípios arquiteturais

- **IA restrita à detecção:** o modelo supervisionado é utilizado exclusivamente para identificar os componentes na imagem. Toda a análise de ameaças é determinística e baseada em regras.
- **Separação de responsabilidades:** interface (Streamlit), pré-processamento, detecção, extração de relacionamentos, engine STRIDE e geração de relatório operam em camadas e módulos distintos.
- **Base de conhecimento desacoplada:** a engine STRIDE não conhece a origem dos dados. No MVP, a base é um arquivo JSON e pode ser atualizada sem alterar o código.
- **Reutilização de código:** o módulo de detecção, o dataset e o modelo treinado foram concebidos para serem reutilizados exatamente como estão, sem retrabalho.
- **Determinismo:** dada a mesma imagem e a mesma base de conhecimento, o sistema produz sempre o mesmo resultado, o que torna o comportamento previsível e auditável.
- **Falhas contidas:** erros de imagem inválida ou de execução são sinalizados por exceções descritivas e tratados na camada de interface, sem derrubar a aplicação.

3.2. Diagrama de arquitetura

O diagrama a seguir apresenta a organização em camadas da solução, evidenciando o fluxo desde a interface até a geração do relatório.

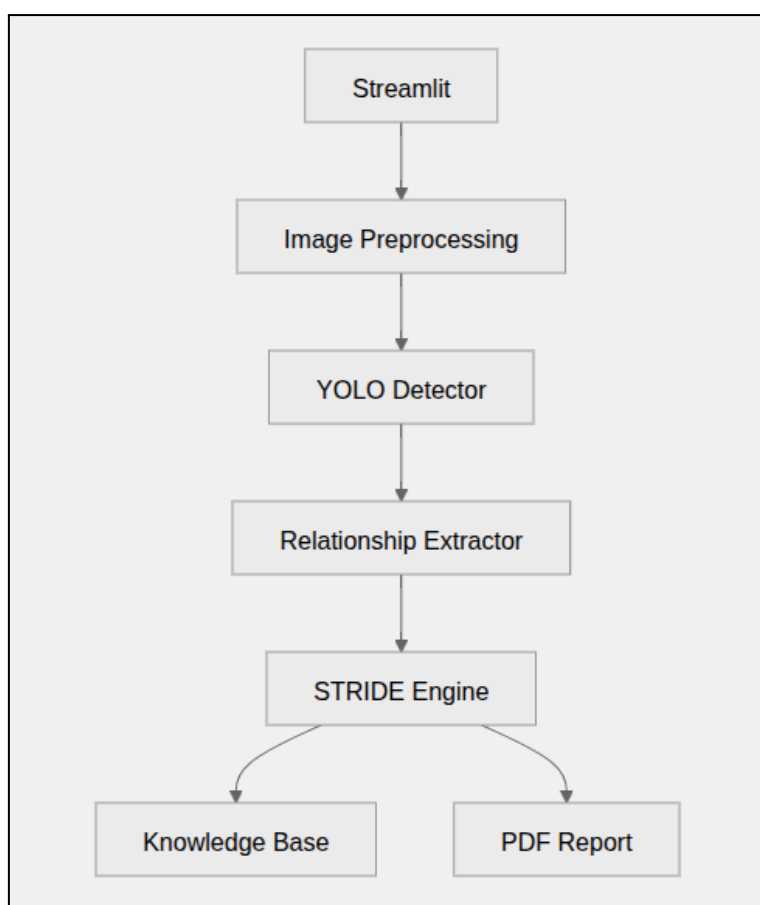


Imagem 1 – Diagrama de arquitetura em camadas da solução

3.3. Estrutura de diretórios

A organização de diretórios evidencia o pipeline completo de IA supervisionada, separando o ciclo de vida do modelo (dataset, treinamento e pesos) do ciclo de vida da aplicação (código-fonte em `src/`).

```
tech_challenge_fiap_fase5/
├── app.py                # Aplicação Streamlit (ponto de entrada)
├── requirements.txt      # Dependências do projeto
├── README.md            # Documentação geral
├── docs/                # Documentação de arquitetura e dataset
├── src/                 # Código-fonte da aplicação
│   ├── preprocessing.py # Pré-processamento da imagem
│   ├── detector.py       # Carregamento do modelo e inferência YOLO
│   ├── relationship_extractor.py # Extração geométrica de relacionamentos
│   ├── stride_engine.py  # Engine de regras que consulta a base JSON
│   ├── report_generator.py # Geração do relatório em PDF (ReportLab)
│   └── utils.py          # Funções auxiliares compartilhadas
├── datasets/
│   ├── stride-architecture-components-v1/ # Dataset anotado (formato YOLO)
│   │   ├── train/ (images, labels)
│   │   ├── val/   (images, labels)
│   │   └── test/  (images, labels)
│   └── data.yaml    # Configuração do dataset (32 classes e splits)
├── runs/
│   └── stride_yololls_baseline/ # Artefatos do treinamento (métricas e curvas)
│       └── weights/best.pt      # Pesos do modelo treinado
├── models/
│   └── best.pt                 # Pesos do modelo em uso pela aplicação
├── data/
│   └── stride_knowledge_base.json # Base de conhecimento STRIDE
├── assets/
│   ├── samples/               # Imagens de exemplo para demonstração
│   └── reports/                # Relatórios das fases anteriores
```

Imagem 2 – Estrutura de diretórios do projeto

4. Fluxo do pipeline

O sistema opera em um fluxo sequencial no qual a imagem enviada percorre todas as etapas até a produção do relatório. Cada etapa possui responsabilidade única e comunica-se com a seguinte por meio de estruturas de dados explícitas.

4.1. Diagrama de fluxo

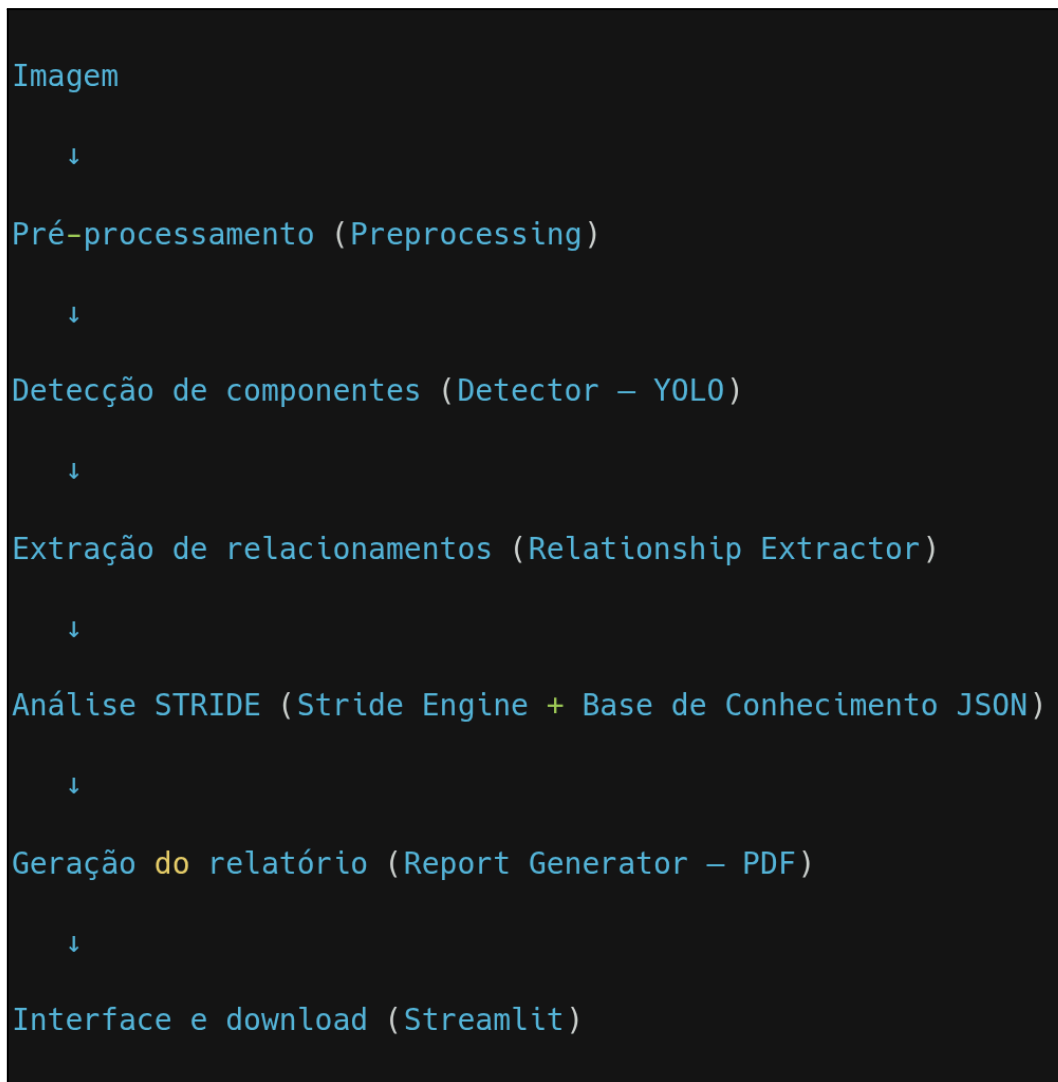


Imagem 3 – Diagrama de fluxo do pipeline completo

4.2. Descrição das etapas

1. **Upload da imagem:** o usuário envia, pela interface Streamlit, a imagem de um diagrama de arquitetura nos formatos PNG, JPG ou JPEG.
2. **Pré-processamento:** a imagem é validada e normalizada para o formato de cor RGB, adequado à inferência. O processamento é intencionalmente mínimo, pois o próprio modelo executa o redimensionamento interno.
3. **Detecção de componentes:** o modelo YOLO treinado executa a inferência e retorna a lista de componentes detectados, cada um com nome da classe, identificador, confiança, *bounding box*, dimensões e centro.
4. **Extração de relacionamentos:** a partir das detecções, o sistema infere relacionamentos entre os componentes utilizando exclusivamente informação geométrica (posição e dimensões das *bounding boxes*), sem qualquer reconhecimento de texto.

5. **Análise STRIDE:** a engine de regras consulta a base de conhecimento JSON e associa a cada componente e relacionamento as ameaças STRIDE correspondentes, com descrição e contramedidas.
6. **Geração do relatório:** o resultado consolidado é transformado em um relatório PDF estruturado, contendo componentes, relacionamentos, ameaças e recomendações.
7. **Apresentação e download:** a interface exibe a imagem com as detecções destacadas, a lista de componentes e as ameaças, além de disponibilizar o download do relatório em PDF.

5. Tecnologias utilizadas

A seleção tecnológica priorizou bibliotecas consolidadas no ecossistema Python de IA e Visão Computacional, com foco em simplicidade, facilidade de integração e adequação a um protótipo acadêmico. As versões correspondem às efetivamente utilizadas no projeto.

Categoria	Tecnologia	Versão	Função
Interface	Streamlit	1.38.0	Interface web interativa
Detecção supervisionada	Ultralytics YOLO	8.2.103	Detecção de componentes em diagramas
Processamento de imagem	OpenCV	4.10.0.84	Leitura e manipulação de imagens
Processamento de imagem	Pillow	10.4.0	Manipulação de imagens e integração com o Streamlit
Computação numérica	NumPy	1.26.4	Operações sobre arrays de imagem
Geração de relatório	ReportLab	4.2.2	Geração programática do PDF
Configuração do dataset	PyYAML	6.0.2	Leitura do arquivo <code>data.yaml</code>
Linguagem	Python	3.10	Base do desenvolvimento

Tabela 1 – Relação de tecnologias utilizadas e suas funções.

5.1. Justificativa das escolhas

- **Streamlit** foi selecionado por permitir a construção de uma interface web funcional utilizando apenas Python, reduzindo a complexidade de desenvolvimento *frontend* e favorecendo demonstrações acadêmicas.
- **Ultralytics YOLO** foi adotado por ser um framework de detecção de objetos consolidado, com bom desempenho, API simples para treino e inferência e forte documentação, além de suportar transfer learning a partir de modelos pré-treinados.
- **OpenCV** e **Pillow** foram utilizados para leitura, conversão e manipulação de imagens, sendo padrões maduros no ecossistema de Visão Computacional.
- **NumPy** dá suporte às operações sobre arrays que representam as imagens e as coordenadas das detecções.
- **ReportLab** foi escolhido por ser uma biblioteca robusta para geração programática de PDFs estruturados.
- **JSON** foi adotado como formato da base de conhecimento STRIDE por ser simples, legível, editável e desacoplado do código.
- **Python 3.10** foi utilizado como linguagem principal por sua ampla compatibilidade com as bibliotecas modernas de IA e Visão Computacional.

Decisão arquitetural: o projeto **não utiliza** LangChain, RAG, bancos vetoriais, LLMs ou agentes de IA. A Inteligência Artificial está restrita ao modelo supervisionado de detecção; toda a modelagem de ameaças é determinística e baseada em regras.

6. Dataset supervisionado

O dataset é o insumo central do treinamento supervisionado. Como o modelo aprende a detectar componentes a partir de exemplos rotulados, a qualidade e a representatividade do dataset determinam diretamente a acurácia da detecção na aplicação final.

6.1. Classes do modelo

O dataset **stride-architecture-components-v1** define **32 classes**, cobrindo atores, componentes de borda, integração, computação, dados, segurança, observabilidade, serviços externos e limites de confiança (*trust boundaries*). A tabela a seguir apresenta as classes por grupo.

Grupo	Classes
Atores	<code>actor_user</code> , <code>actor_admin</code>
Borda (edge)	<code>edge_ddos_protection</code> , <code>edge_cdn</code> , <code>edge_waf</code> , <code>edge_gateway</code> , <code>edge_portal</code> , <code>external_entry_point</code>
Integração	<code>integration_orchestrator</code> , <code>integration_messaging</code>

Grupo	Classes
Computação	<code>compute_load_balancer</code> , <code>compute_service</code> , <code>compute_worker</code>
Dados	<code>data_database</code> , <code>data_cache</code> , <code>data_storage</code>
Segurança	<code>security_identity_provider</code> , <code>security_key_management</code>
Observabilidade	<code>obs_monitoring</code> , <code>obs_audit</code>
Serviços externos	<code>external_backend_service</code> , <code>external_saas_service</code> , <code>external_web_service</code> , <code>communication_service</code> , <code>backup_service</code>
Limites de confiança	<code>boundary_cloud</code> , <code>boundary_region</code> , <code>boundary_resource_group</code> , <code>boundary_vpc_or_vnet</code> , <code>boundary_subnet_public</code> , <code>boundary_subnet_private</code> , <code>boundary_autoscaling_group</code>

Tabela 2 – Classes do dataset agrupadas por categoria arquitetural.

A inclusão das classes com prefixo `boundary_` é uma característica relevante do projeto: os limites de confiança são fundamentais na metodologia STRIDE, pois delimitam regiões de confiança e influenciam diretamente a análise de ameaças em trânsito e de escalonamento de privilégios.

6.2. Fontes e coleta das imagens

As imagens foram coletadas a partir de diagramas de arquitetura publicamente disponíveis, incluindo:

- Arquiteturas de referência da AWS;
- Arquiteturas de referência da Azure;
- Publicações de blogs sobre soluções em nuvem;
- Exemplos de documentação técnica.

O dataset foca deliberadamente em diagramas de arquitetura de nuvem (AWS e Azure), o que confere coerência semântica ao conjunto de classes e aproxima o modelo de cenários reais de análise de segurança.

6.3. Estratégia de *data augmentation*

Para aumentar a robustez do modelo e simular variações do mundo real, foram aplicadas técnicas de *data augmentation* sobre as imagens originais. Cada variação é identificada por um sufixo no nome do arquivo:

Sufixo	Transformação
_bw	Conversão para preto e branco
_sharp	Filtro de nitidez
_contrast	Ajuste de contraste
_gamma_hi	Correção de gama alta
_gamma_lo	Correção de gama baixa
_jpeg50	Compressão JPEG a 50%
_blur1	Desfoque gaussiano (nível 1)
_noise6	Injeção de ruído (nível 6)
_degrade80	Degradação de qualidade (80%)

Tabela 3 – Técnicas de aumento de dados aplicadas ao dataset.

Essas transformações forçam o modelo a generalizar a **forma e a semântica** dos componentes, em vez de memorizar condições ideais de imagem, tornando a detecção mais tolerante a variações de qualidade, iluminação e compressão.

6.4. Divisão do dataset

O dataset foi organizado no padrão YOLO, com três conjuntos independentes — treino, validação e teste — cada um com suas respectivas pastas de imagens e rótulos.

Conjunto	Imagens	Finalidade
Treino (train)	2.980	Ajuste dos pesos do modelo
Validação (val)	840	Monitoramento do desempenho durante o treino
Teste (test)	230	Avaliação final independente

Tabela 4 – Distribuição das imagens entre os conjuntos do dataset.

A divisão em três conjuntos permite acompanhar o desempenho durante o treinamento (validação) e obter uma medida imparcial em imagens não vistas (teste), tornando as métricas reportadas confiáveis e defensáveis.

6.5. Formato de anotação

As anotações foram criadas em **formato YOLO**, no qual cada imagem possui um arquivo **.txt** de mesmo nome-base contendo uma linha por objeto:

<class_id> <x_center> <y_center> <width> <height>

Todas as coordenadas são normalizadas entre 0 e 1 em relação às dimensões da imagem. A definição das 32 classes e dos caminhos dos *splits* é registrada no arquivo **data.yaml**.

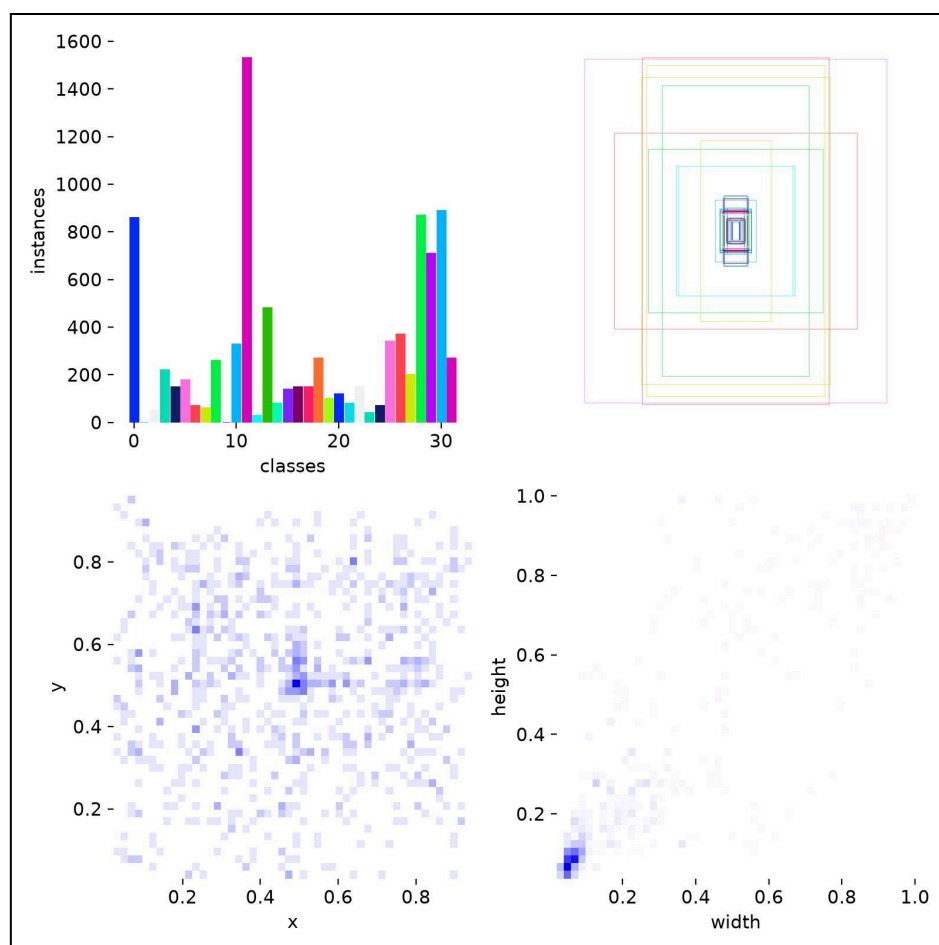


Imagem 4 – Distribuição de instâncias por classe no dataset

7. Treinamento do modelo

O modelo de detecção foi produzido por meio de treinamento supervisionado, utilizando transfer learning a partir de um modelo pré-treinado da família YOLO.

7.1. Configuração e hiperparâmetros

O treinamento utilizou o modelo base **YOLO11s** ([yolo11s.pt](#)), ajustado ao dataset de diagramas de arquitetura. A tabela a seguir resume os principais hiperparâmetros efetivamente utilizados.

Parâmetro	Valor
Modelo base	yolo11s.pt (pré-treinado)
Tarefa	Detecção de objetos (detect)
Épocas	100
Tamanho da imagem	640 × 640
Otimizador	auto
Taxa de aprendizado inicial (lr0)	0.01
Fator de taxa de aprendizado final (lrf)	0.01
Momentum	0.937
Weight decay	0.0005
Épocas de <i>warmup</i>	3
<i>Mixed precision</i> (AMP)	Ativado
Semente (seed)	42
Determinístico	Sim

Tabela 5 – Principais hiperparâmetros do treinamento.

A adoção de semente fixa ([seed](#) = 42) e do modo determinístico favorece a reprodutibilidade do experimento. A técnica de *close_mosaic* foi aplicada nas 10 épocas finais, desativando a composição em mosaico para estabilizar o aprendizado na reta final do treinamento.

7.2. Ambiente de treinamento

O treinamento foi conduzido em ambiente com aceleração por GPU (*device 0*), com *mixed precision* habilitado para otimizar o uso de memória e o tempo de processamento. O tempo total de treinamento das 100 épocas foi de aproximadamente **47 minutos**.

7.3. Evolução do treinamento

As curvas de perda (*box loss*, *classification loss* e *distribution focal loss*) e as métricas de validação ao longo das épocas evidenciam a convergência do modelo. Observa-se queda

consistente das perdas de treino e estabilização das métricas de validação a partir da metade do treinamento.

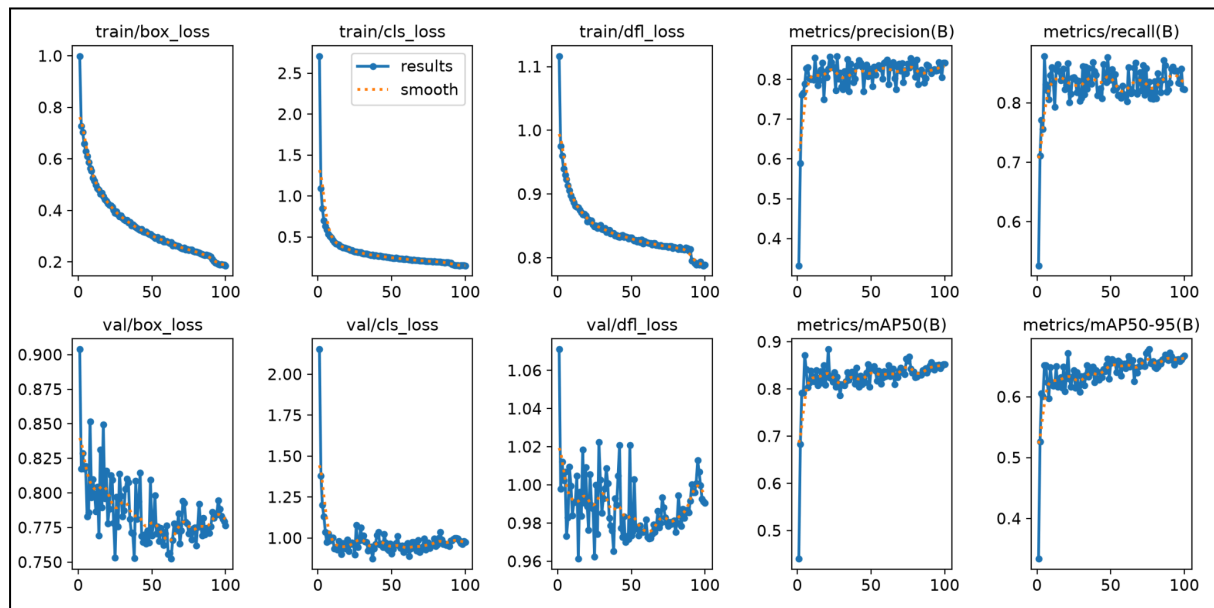


Imagem 5 – Curvas de treinamento e validação ao longo das épocas

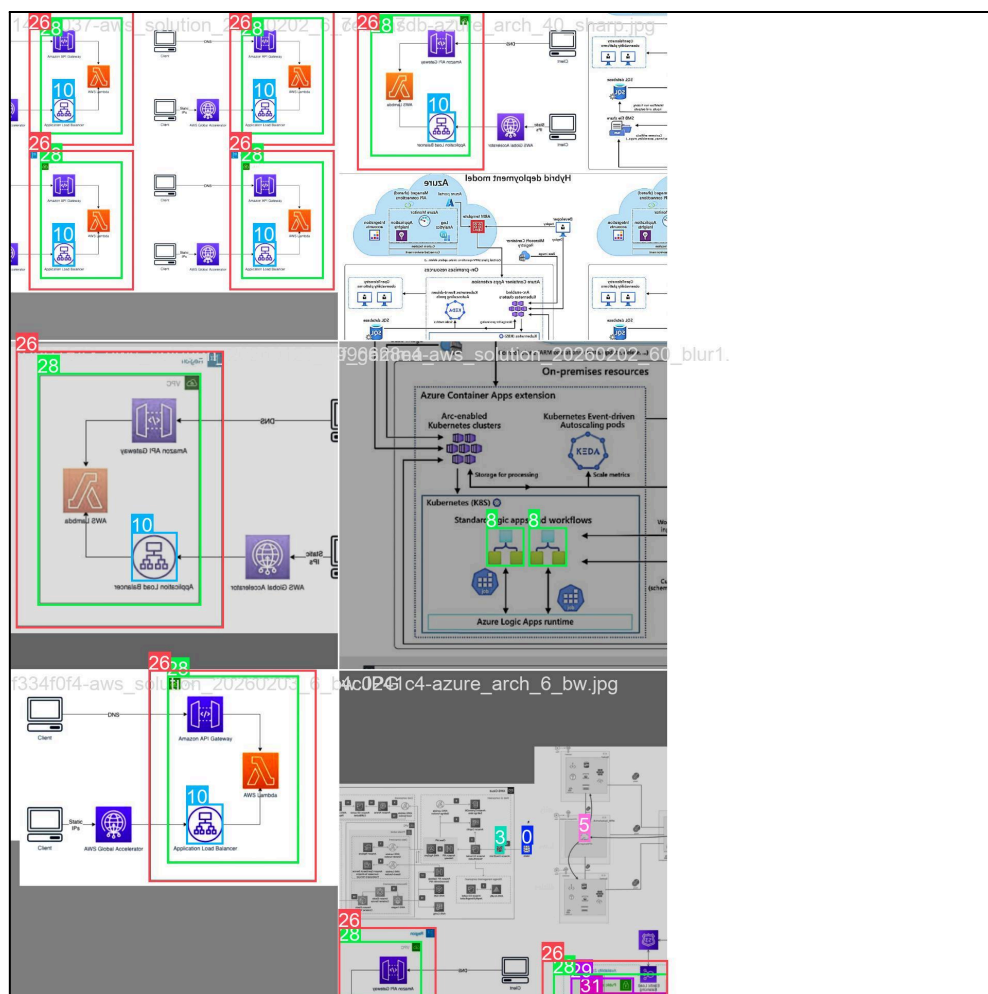


Imagem 6 – Exemplo de lote de treinamento com aumento de dados aplicado

8. Avaliação do modelo

A avaliação foi realizada sobre o conjunto de validação, utilizando as métricas padrão de detecção de objetos: precisão, *recall* e *mean Average Precision* (mAP) nos limiares de IoU de 0,50 (mAP50) e no intervalo de 0,50 a 0,95 (mAP50-95).

8.1. Métricas finais

Métrica	Valor (época final)	Melhor valor
Precisão (<i>Precision</i>)	0,842	—
Revocação (<i>Recall</i>)	0,823	—
mAP@50	0,852	0,868
mAP@50-95	0,667	0,678

Tabela 6 – Métricas de avaliação do modelo no conjunto de validação.

O modelo alcançou **mAP@50 de aproximadamente 0,85** e **mAP@50-95 de aproximadamente 0,67**, resultados consistentes para um problema com 32 classes e diagramas de estilos visuais heterogêneos. A precisão de 0,84 e o *recall* de 0,82 indicam um bom equilíbrio entre a taxa de detecções corretas e a cobertura dos componentes presentes nos diagramas.

8.2. Curvas de desempenho

As curvas de Precisão × Confiança, *Recall* × Confiança, F1 × Confiança e Precisão × *Recall* detalham o comportamento do modelo em diferentes limiares de confiança, apoiando a escolha de um limiar adequado para a inferência.

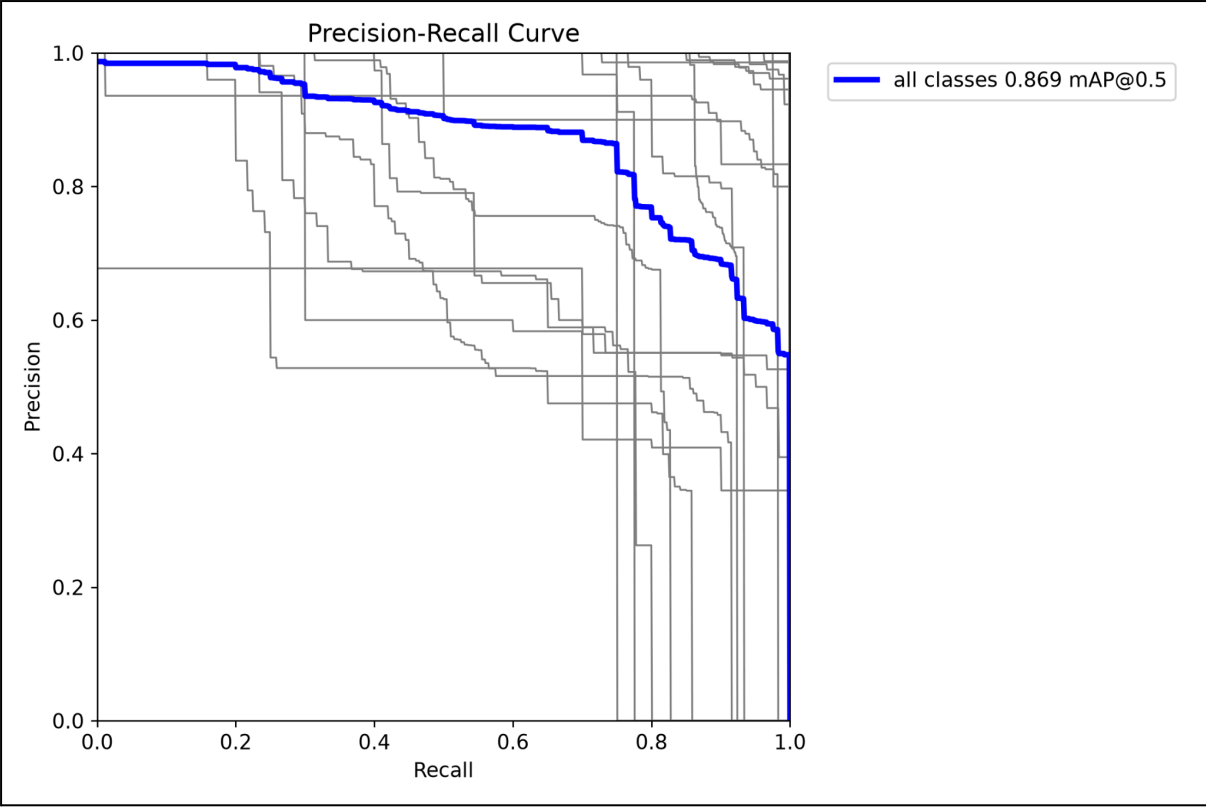


Imagem 7 – Curva Precisão × Recall por classe

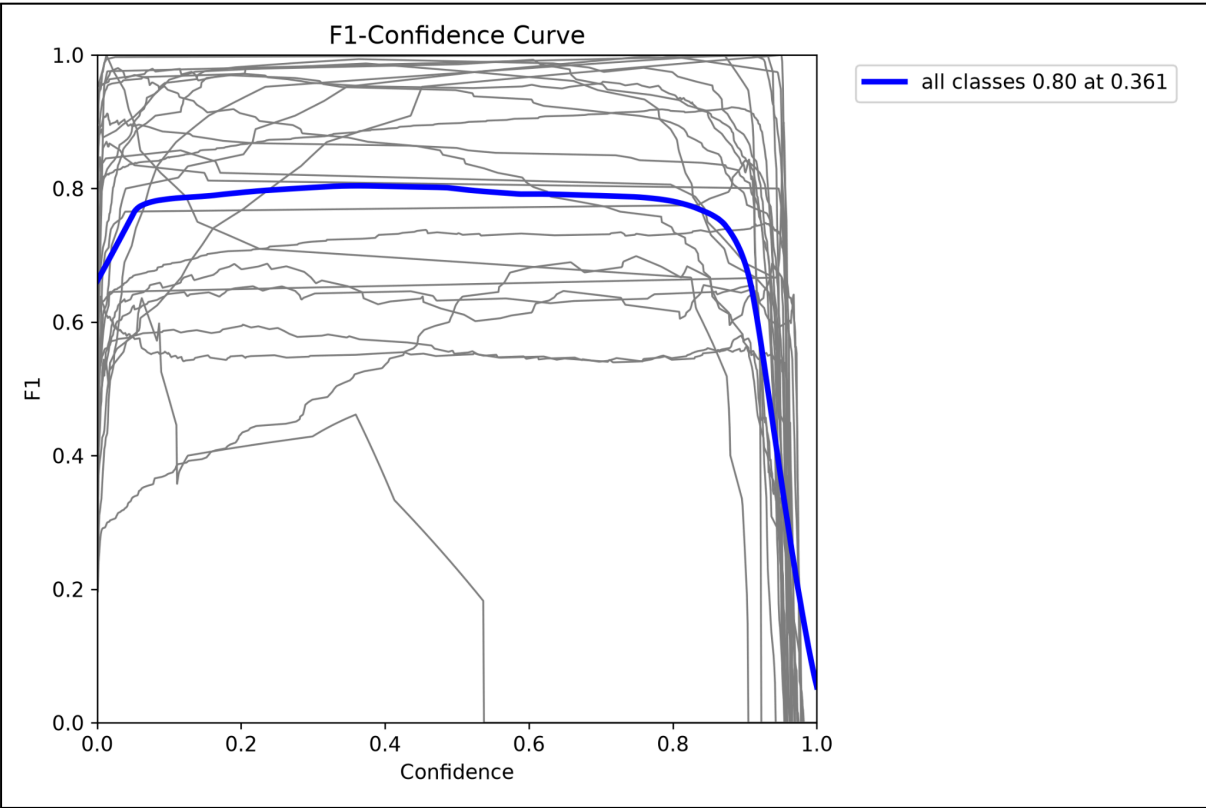


Imagem 8 – Curva F1 × Confiança

8.3. Matriz de confusão

A matriz de confusão evidencia o desempenho por classe e as principais confusões entre categorias visualmente semelhantes, informação útil para futuras iterações de anotação e balanceamento do dataset.

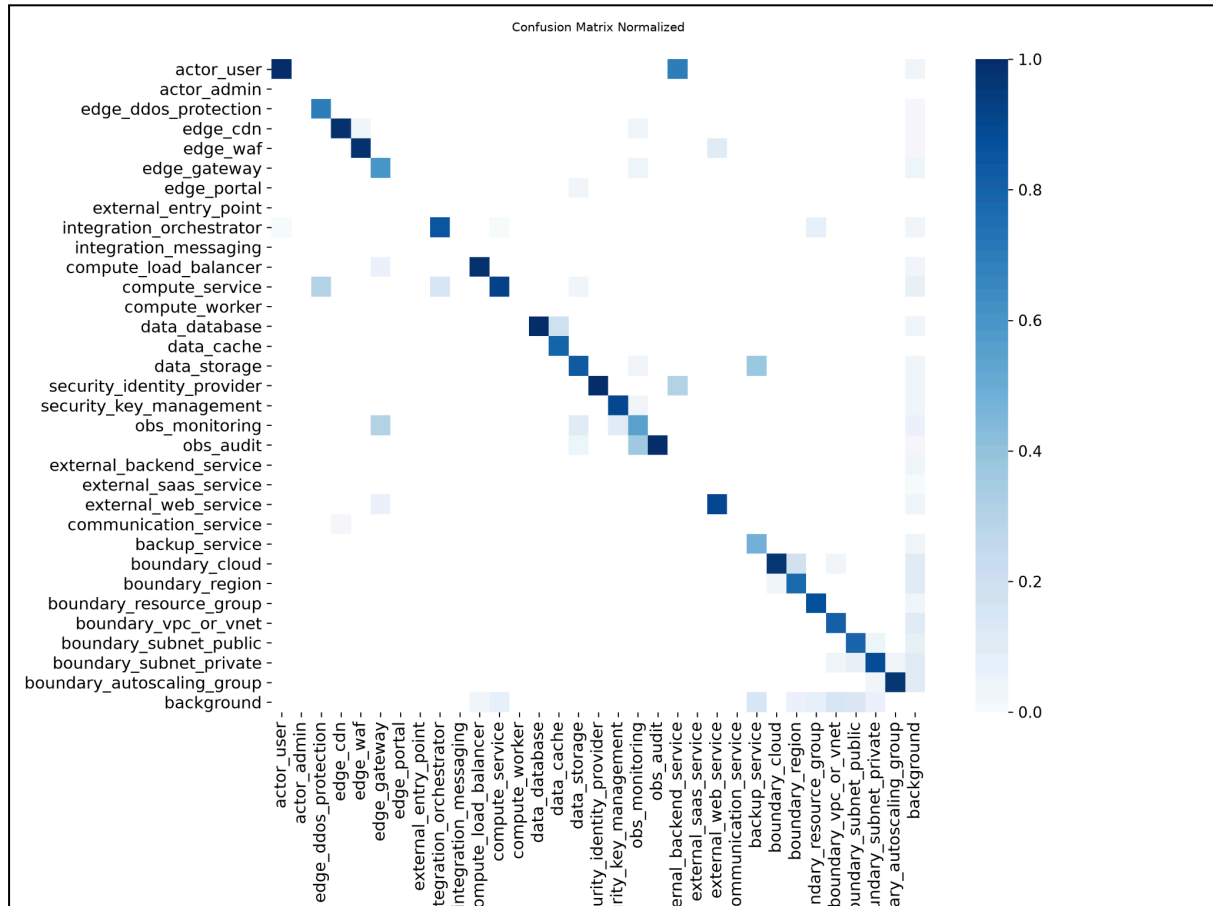


Imagem 9 – Matriz de confusão normalizada

8.4. Comparação entre anotações e predições

A comparação visual entre os rótulos reais e as predições do modelo em lotes do conjunto de validação demonstra qualitativamente a capacidade de detecção do modelo em diagramas completos.

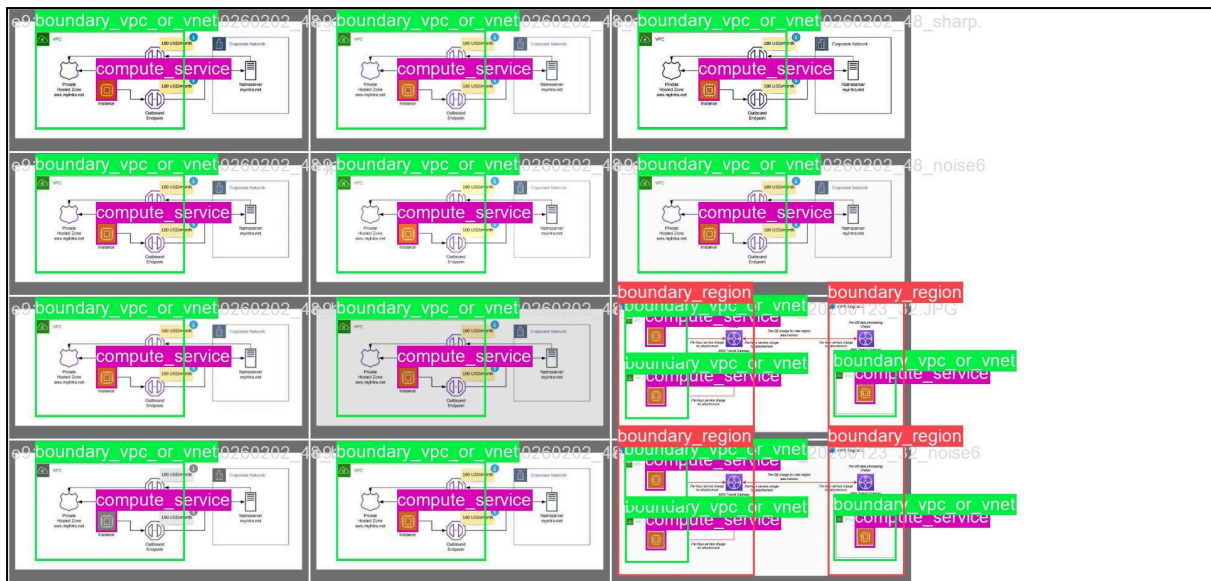


Imagem 10 – Rótulos reais de um lote de validação

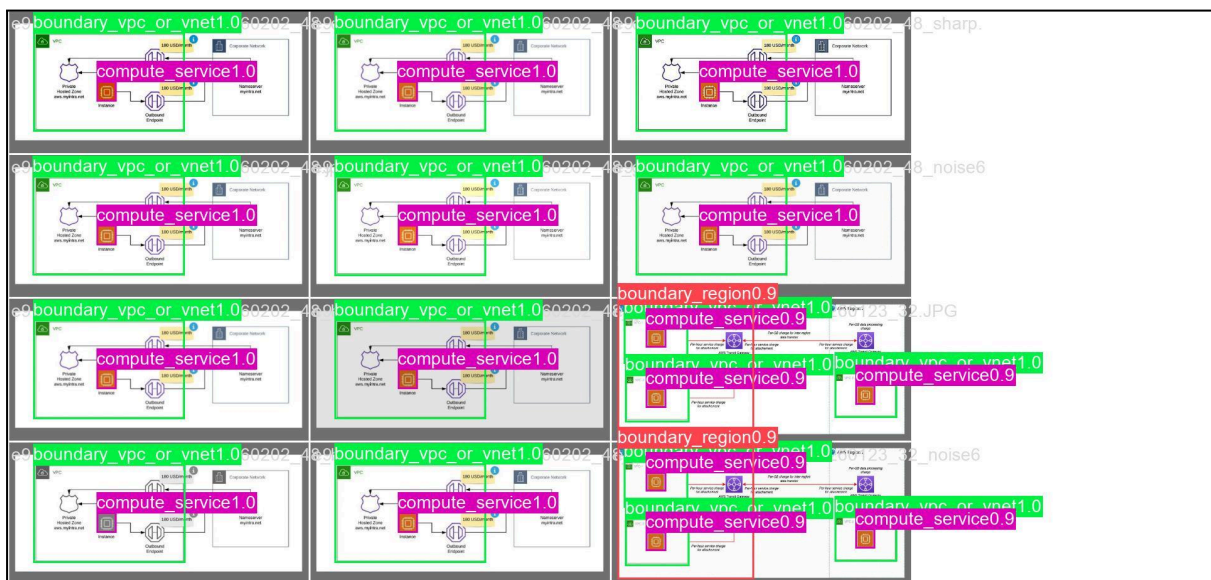


Imagem 11 – Predições do modelo no mesmo lote de validação

9. Módulos da aplicação

A aplicação foi organizada em módulos com responsabilidades bem delimitadas, refletindo as etapas do pipeline. Esta seção detalha cada um deles.

9.1. Detector (`src/detector.py`)

O módulo `detector.py` encapsula completamente o uso do modelo YOLO. Ele carrega os pesos treinados (`models/best.pt`) uma única vez, durante a inicialização, e os reutiliza em todas as chamadas de inferência. A classe `Detector` expõe o método `detect`, que recebe uma imagem (caminho de arquivo, array NumPy ou objeto Pillow) e retorna a lista de componentes detectados.

Cada detecção é representada por uma estrutura de dados com os seguintes campos:

Campo	Descrição
<code>class_name</code>	Nome da classe detectada
<code>class_id</code>	Índice numérico da classe
<code>confidence</code>	Confiança da detecção (0,0 a 1,0)
<code>bbox</code>	Caixa delimitadora [<code>x1</code> , <code>y1</code> , <code>x2</code> , <code>y2</code>] em pixels
<code>width</code>	Largura da caixa em pixels
<code>height</code>	Altura da caixa em pixels
<code>center</code>	Centro da caixa (<code>cx</code> , <code>cy</code>) em pixels

Tabela 7 – Estrutura de dados de uma detecção.

O módulo é totalmente desacoplado da interface e da lógica STRIDE, podendo ser reutilizado por qualquer parte da aplicação. Também implementa tratamento de erros para os casos de modelo ausente ou falha de inferência.

9.2. Pré-processamento (`src/preprocessing.py`)

O módulo de pré-processamento tem a responsabilidade única de preparar a imagem enviada para a inferência. Recebe a entrada em diferentes formatos (bytes do upload, caminho, array NumPy ou objeto Pillow), valida que se trata de uma imagem legível e a normaliza para o modo de cor RGB, aceito diretamente pelo Detector.

O processamento é intencionalmente **mínimo**: não são aplicados redimensionamentos agressivos nem filtros, uma vez que o próprio modelo YOLO executa o redimensionamento interno. Caso a entrada seja inválida ou não possa ser decodificada como imagem, o módulo sinaliza um erro descritivo por meio de uma exceção dedicada (`InvalidImageError`).

9.3. Extração de relacionamentos (`src/relationship_extractor.py`)

Este módulo é responsável por inferir como os componentes detectados se relacionam entre si, utilizando **exclusivamente informação geométrica** das *bounding boxes* e de seus centros — sem qualquer reconhecimento de texto (OCR). O objetivo não é reconstruir perfeitamente o diagrama, mas identificar relações suficientes para a análise STRIDE.

São inferidos dois tipos de relacionamento por meio de heurísticas geométricas:

- **Containment (*contains*)**: relaciona um limite de confiança (*trust boundary*, classes com prefixo *boundary_*) aos componentes que estão geometricamente contidos em sua região. Esse relacionamento é essencial para a análise STRIDE, pois delimita regiões de confiança.
- **Conexão/vizinhança (*connected_to*)**: relaciona componentes próximos entre si, representando conexões inferidas a partir da proximidade, suficientes para que a engine STRIDE avalie a exposição entre componentes.

Cada relacionamento é representado por uma estrutura simples contendo o tipo e os dois componentes envolvidos, mantendo baixo acoplamento com as demais camadas. Quando não há componentes, o módulo retorna uma lista vazia.

9.4. Engine STRIDE (*src/stride_engine.py*)

A engine STRIDE recebe os componentes detectados e os relacionamentos inferidos e consulta a base de conhecimento JSON para produzir a lista de ameaças. A análise é **100% baseada em regras**, determinística e sem qualquer uso de Inteligência Artificial.

O funcionamento é o seguinte:

- Para cada componente cujo nome de classe possui entradas na base de conhecimento, é gerada uma ameaça por entrada, contendo a categoria STRIDE, a descrição e as contramedidas.
- Para cada relacionamento cujo tipo possui regras na base, são geradas ameaças adicionais associadas ao par de componentes.
- Componentes sem correspondência na base simplesmente não geram ameaças, sem interromper a análise dos demais.

A base de conhecimento é carregada uma única vez, durante a inicialização da engine, e o resultado é ordenado de forma estável, garantindo que a mesma entrada produza sempre a mesma saída.

Cada ameaça é representada pela seguinte estrutura:

Campo	Descrição
<i>component</i>	Nome do componente afetado
<i>category</i>	Categoria STRIDE
<i>description</i>	Descrição da ameaça
<i>countermeasures</i>	Lista de contramedidas recomendadas
<i>related_to</i>	Componente relacionado (em regras de relacionamento) ou vazio

Tabela 8 – Estrutura de dados de uma ameaça identificada.

9.5. Gerador de relatório (`src/report_generator.py`)

O módulo de geração de relatório utiliza a biblioteca ReportLab para produzir um PDF estruturado a partir dos componentes, relacionamentos e ameaças. O relatório contém, no mínimo:

- Cabeçalho com título e data de geração;
- Seção com os **componentes detectados** (nome, confiança e *bounding box*);
- Seção com os **relacionamentos** encontrados, no formato `tipo: origem → destino`;
- Seção com as **ameaças e contramedidas**, agrupadas por componente e categoria STRIDE;
- Indicação explícita quando nenhuma ameaça é identificada.

A função principal retorna os *bytes* do PDF, que alimentam diretamente o botão de download da interface, e, opcionalmente, grava o arquivo em disco.

9.6. Utilitários (`src/utils.py`)

O módulo de utilitários reúne funções auxiliares compartilhadas, mantidas ao mínimo necessário:

- `load_json`: carregamento seguro de arquivos JSON (utilizado pela engine STRIDE);
- `draw_detections`: desenho das *bounding boxes* e dos rótulos das classes sobre uma cópia da imagem, preservando a original, para exibição das detecções na interface.

9.7. Aplicação Streamlit (`app.py`)

A aplicação Streamlit integra todo o pipeline e fornece a interface ao usuário. O Detector e a engine STRIDE são instanciados uma única vez por sessão, por meio do mecanismo de cache de recursos do Streamlit (`@st.cache_resource`), evitando o recarregamento do modelo e da base a cada interação.

O fluxo da interface, ao receber uma imagem, é o seguinte:

1. Pré-processa a imagem enviada;
2. Executa a detecção de componentes;
3. Caso nenhum componente seja detectado, informa o usuário e interrompe o fluxo de forma controlada;
4. Extrai os relacionamentos e executa a análise STRIDE;
5. Exibe a imagem com as detecções destacadas por *bounding boxes* e rótulos;
6. Apresenta a lista de componentes detectados e as ameaças identificadas, agrupadas por componente;
7. Gera o relatório em PDF e disponibiliza o botão de download.

Todo o fluxo é protegido por tratamento de exceções: qualquer falha (imagem inválida, erro de carregamento do modelo ou de inferência) é apresentada por meio de uma mensagem descritiva, sem derrubar a aplicação.



Imagem 12 – Interface inicial da aplicação com o componente de upload

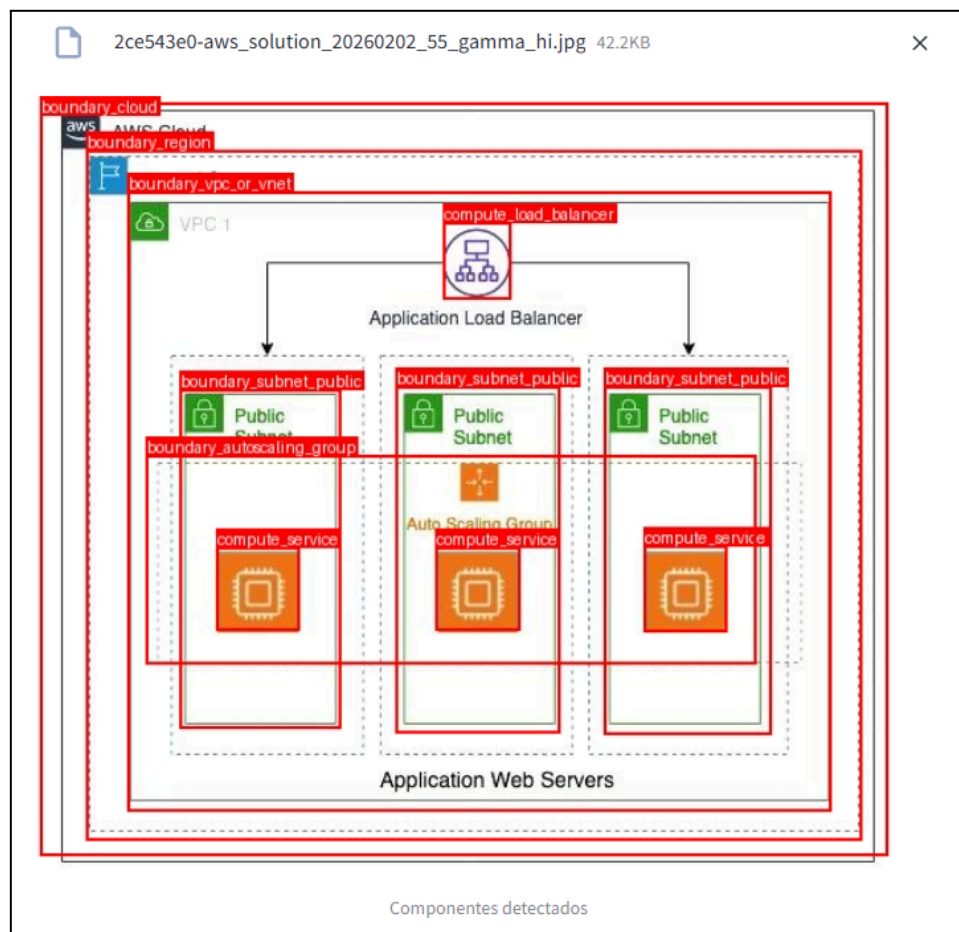


Imagem 13 – Imagem do diagrama com os componentes detectados destacados por bounding boxes e rótulos

Componente	Confiança
boundary_region	97.4%
boundary_vpc_or_vnet	97.2%
boundary_cloud	94.1%
compute_service	93.7%
compute_service	93.5%
compute_service	93.1%
boundary_subnet_public	93.0%
boundary_subnet_public	92.8%
boundary_subnet_public	92.7%
boundary_autoscaling_group	89.0%

Imagem 14 – Lista de componentes detectados e ameaças STRIDE identificadas na interface

10. Base de conhecimento STRIDE

A base de conhecimento é o coração da análise de ameaças. Trata-se de um arquivo JSON ([data/stride_knowledge_base.json](#)) desacoplado do código, o que permite atualizá-la e expandi-la sem qualquer alteração na lógica da aplicação.

10.1. Estrutura

A base é organizada em dois blocos principais:

- **components:** mapeia cada tipo de componente (pelo nome de classe produzido pelo Detector) a uma lista de ameaças. Cada ameaça contém a categoria STRIDE, uma descrição e uma lista de contramedidas.
- **relationship_rules:** mapeia cada tipo de relacionamento ([connected_to](#), [contains](#)) a uma lista de ameaças, na mesma estrutura, permitindo que a análise leve em conta as relações entre componentes.

As categorias utilizadas são restritas às seis da metodologia STRIDE: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service* e *Elevation of Privilege*.

10.2. Exemplo de entrada

O trecho a seguir ilustra a estrutura da base para o componente de banco de dados:

```

{
  "components":{
    "data_database":[
      {
        "category":"Tampering",
        "description":"Dados persistidos podem ser alterados indevidamente
por acesso não autorizado ou por ataques de injeção SQL.",
        "countermeasures":[
          "Aplicar controle de acesso baseado em papéis (RBAC) e menor
privilégio.",
          "Usar consultas parametrizadas para evitar injeção de SQL.",
          "Habilitar auditoria e trilhas de alteração."
        ]
      }
    ]
  },
  "relationship_rules":{
    "connected_to":[
      {
        "category":"Information Disclosure",
        "description":"A comunicação entre componentes conectados pode
expor dados em trânsito se o canal não for protegido.",
        "countermeasures":[
          "Usar TLS/mTLS entre os componentes conectados.",
          "Autenticar as duas pontas da conexão."
        ]
      }
    ]
  }
}

```

Imagem 15 - Estrutura da base para o componente de banco de dados

10.3. Cobertura

No MVP, a base de conhecimento foi populada com os **componentes de maior risco**, cobrindo atores, componentes de borda, computação, dados, segurança, integração e serviços externos, além de regras para os relacionamentos de conexão e containment. Não é necessário cobrir todas as 32 classes: componentes sem entrada na base simplesmente não geram ameaças, o que mantém o sistema robusto e permite a expansão incremental da cobertura.

11. Resultados obtidos

Os resultados apresentados nesta seção demonstram o funcionamento *end-to-end* do sistema, do upload da imagem à geração do relatório.

11.1. Detecção de componentes

O modelo YOLO11s treinado detecta os componentes arquiteturais nos diagramas com bom desempenho, conforme as métricas apresentadas na Seção 8 (mAP@50 de aproximadamente 0,85). Na interface, a imagem enviada é exibida com as *bounding boxes* e os rótulos de cada componente detectado, permitindo a conferência visual do resultado durante a demonstração.

11.2. Extração de relacionamentos

A partir das detecções, o sistema infere os relacionamentos geométricos entre os componentes. Em um diagrama contendo um limite de confiança (por exemplo, uma VPC/VNet) com um *gateway*, um serviço de computação e um banco de dados em seu interior, o sistema identifica corretamente as relações de containment entre o limite e os componentes internos, bem como as conexões de vizinhança entre os componentes próximos.

11.3. Análise STRIDE

A engine associa a cada componente e relacionamento as ameaças correspondentes da base de conhecimento. Por consultar uma base determinística, a análise é totalmente reproduzível: a mesma imagem gera sempre o mesmo conjunto de ameaças. As ameaças são apresentadas na interface agrupadas por componente, com sua categoria STRIDE, descrição e contramedidas.

11.4. Relatório em PDF

Ao final, o sistema gera um relatório em PDF estruturado, contendo os componentes detectados, os relacionamentos, as ameaças identificadas e as recomendações. O relatório é disponibilizado para download diretamente na interface, oferecendo um artefato consolidado para apoio à análise de segurança.

Relatório de Análise STRIDE

Gerado em 26/07/2026 10:38

Componentes detectados

Componente	Confiança	Bounding box [x1, y1, x2, y2]
backup_service	97.8%	[1121, 359, 1214, 455]
boundary_vpc_or_vnet	97.2%	[603, 75, 1267, 317]
boundary_cloud	96.5%	[166, 8, 1301, 518]
edge_waf	96.4%	[425, 125, 518, 221]
boundary_region	96.1%	[376, 55, 1300, 343]
obs_monitoring	95.6%	[422, 358, 520, 453]
actor_user	94.6%	[34, 118, 121, 224]
compute_load_balancer	93.2%	[656, 122, 750, 223]

Relacionamentos

- contains: boundary_vpc_or_vnet -> compute_load_balancer
- contains: boundary_cloud -> backup_service
- contains: boundary_cloud -> edge_waf
- contains: boundary_cloud -> obs_monitoring
- contains: boundary_cloud -> compute_load_balancer
- contains: boundary_region -> edge_waf
- contains: boundary_region -> compute_load_balancer

Ameaças e contramedidas

Componente: actor_user

Repudiation

Um usuário pode negar ter realizado uma ação caso não existam registros confiáveis de auditoria.

Contramedidas:

- Registrar ações relevantes em trilhas de auditoria imutáveis.
- Associar cada ação a uma identidade autenticada e a um carimbo de tempo confiável.

Spoofing

Um atacante pode se passar por um usuário legítimo utilizando credenciais roubadas, reutilizadas ou obtidas por phishing.

Contramedidas:

- Exigir autenticação multifator (MFA) para acesso a recursos sensíveis.
- Adotar políticas de senha forte e detecção de credenciais vazadas.
- Monitorar tentativas de login anômalas e aplicar bloqueio progressivo.

Componente: backup_service

Information Disclosure

Backups armazenam cópias completas de dados sensíveis e são alvo atrativo caso estejam desprotegidos.

Imagem 16 – Exemplo de relatório PDF gerado pela aplicação

11.5. Validação funcional do pipeline

O pipeline foi validado de ponta a ponta, confirmando o encadeamento correto entre a extração de relacionamentos, a engine STRIDE e a geração do relatório. Os testes funcionais confirmaram: a inferência de relacionamentos de containment e de vizinhança; o determinismo da análise STRIDE; a geração de um PDF válido; e o tratamento adequado do caso em que nenhum componente é detectado.

12. Limitações

Como protótipo acadêmico, o sistema possui limitações conhecidas, detalhadas na tabela a seguir.

Limitação	Impacto	Mitigação possível
Detecção restrita a componentes (sem setas/linhas)	O sistema não lê as conexões explícitas do diagrama	Detecção de setas e fluxos em versões futuras
Relacionamentos inferidos por geometria	Conexões podem não corresponder exatamente às do diagrama	Ajuste das heurísticas ou detecção explícita de conexões
Ausência de OCR	Rótulos textuais internos ao diagrama não são interpretados	Introdução de OCR em iterações futuras
Base de conhecimento parcial	Componentes sem entrada não geram ameaças	Expansão incremental da base JSON
Confusão entre classes visualmente semelhantes	Possíveis erros de classificação	Ampliação e balanceamento do dataset
Dataset centrado em AWS e Azure	Menor generalização para outros estilos de diagrama	Inclusão de diagramas de outras fontes e ferramentas

Tabela 9 – Limitações do MVP e mitigações possíveis.

É importante destacar que o sistema é uma **ferramenta de apoio**: os resultados são indicativos e devem ser interpretados por um profissional de segurança, dentro do contexto completo da arquitetura analisada.

13. Melhorias futuras

As melhorias estão organizadas por horizonte de implementação.

13.1. Curto prazo

- **Detecção de conexões:** identificar setas e fluxos entre componentes para uma modelagem STRIDE mais rica, contemplando ameaças em trânsito.

- **Expansão da base de conhecimento:** aumentar a cobertura de componentes e o detalhamento das contramedidas na base JSON.
- **Ajuste do limiar de confiança:** permitir a configuração do limiar de confiança da detecção diretamente na interface.

13.2. Médio prazo

- **OCR de rótulos:** ler textos internos aos diagramas para refinar a identificação e a desambiguação dos componentes.
- **Refinamento das heurísticas de relacionamento:** incorporar critérios adicionais (alinhamento, direção) para aproximar os relacionamentos inferidos das conexões reais.
- **Ampliação do dataset:** incluir diagramas de outras fontes, ferramentas e estilos visuais, aumentando a generalização do modelo.

13.3. Longo prazo

- **Histórico de análises:** armazenar análises anteriores e permitir a comparação entre versões de um mesmo diagrama.
- **Exportação em outros formatos:** gerar relatórios em HTML ou DOCX, além do PDF.
- **Dashboard analítico:** oferecer uma visão consolidada de riscos por projeto ou diagrama.
- **Disponibilização como serviço web:** hospedar a aplicação para acesso remoto.

14. Conclusão

O presente trabalho demonstrou a viabilidade técnica de uma aplicação que integra **Visão Computacional supervisionada e análise de segurança arquitetural** baseada em regras. A solução recebe a imagem de um diagrama de arquitetura, detecta automaticamente seus componentes por meio de um modelo YOLO11s treinado sobre um dataset de 32 classes, infere relacionamentos geométricos entre os componentes, aplica a metodologia STRIDE a partir de uma base de conhecimento determinística e gera um relatório consolidado em PDF.

Os principais resultados alcançados foram:

1. **Pipeline funcional end-to-end:** o sistema processa a imagem e produz o relatório sem intervenção manual entre as etapas.
2. **Modelo de detecção com bom desempenho:** o YOLO11s alcançou mAP@50 de aproximadamente 0,85 sobre 32 classes de componentes e limites de confiança.
3. **Separação clara entre IA e regras:** a Inteligência Artificial atua exclusivamente na detecção, enquanto a modelagem de ameaças permanece determinística, transparente e auditável.
4. **Baixo acoplamento e modularidade:** cada etapa do pipeline é encapsulada em um módulo com responsabilidade única, favorecendo manutenção e evolução independentes.

5. **Reutilização e simplicidade:** o projeto reaproveitou integralmente o módulo de detecção, o dataset e o modelo treinado, mantendo a complexidade proporcional ao contexto acadêmico.

O valor central da proposta reside em **sistematizar a modelagem de ameaças** — atividade tradicionalmente manual e dependente da leitura visual de diagramas — combinando o reconhecimento automático de componentes com uma análise de segurança estruturada e reproduzível. As limitações identificadas são conhecidas e endereçáveis nas iterações futuras descritas na Seção 13.

Como protótipo acadêmico, o sistema cumpre seu objetivo de explorar a convergência entre Visão Computacional e segurança arquitetural, entregando uma ferramenta de apoio que pode contribuir para a adoção de práticas de *security by design*, desde que utilizada com consciência de suas limitações e sob a supervisão de profissionais qualificados.

15. Referências

JOCHER, G.; QIU, J.; CHAURASIA, A. **Ultralytics YOLO**. Ultralytics, 2023. Disponível em: <https://github.com/ultralytics/ultralytics>

MICROSOFT. **The STRIDE Threat Model**. Microsoft Docs, 2009. Disponível em: <https://learn.microsoft.com/security/>

SHOSTACK, A. **Threat Modeling: Designing for Security**. Wiley, 2014.

STREAMLIT. **Streamlit Documentation**. 2024. Disponível em: <https://streamlit.io/>

OPENCV. **OpenCV Documentation**. 2024. Disponível em: <https://opencv.org/>

REPORTLAB. **ReportLab PDF Library**. 2024. Disponível em: <https://www.reportlab.com/>

AMAZON WEB SERVICES. **AWS Architecture Reference**. Disponível em: <https://aws.amazon.com/architecture/>

MICROSOFT. **Azure Architecture Center**. Disponível em: <https://learn.microsoft.com/azure/architecture/>